

Reviewer Pack — Lyndon Factor Width Lower-Bounds Tree-Like Resolution

Entry d04fdfb9056c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 22:08:22 UTC

Lyndon Factor Width Lower-Bounds Tree-Like Resolution

Entry ID: d04fdfb9056c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-07 22:08:22 UTC

1. Conjecture

Field A (mathematical branch): Free Lie algebra combinatorics (Chen-Fox-Lyndon factorization, Witt’s necklace formula) **Field B** (complexity object): Tree-like Resolution / sub-Frege refutation length (gateway to EF lower bounds)

Statement:

Fix an unsatisfiable CNF φ on n variables and a literal ordering $x_1 < \neg x_1 < x_2 < \neg x_2 < \dots < x_n < \neg x_n$; encode each clause C as the indicator vector $b(C) \in \{0,1\}^{\{2n\}}$, and let $L(C)$ be the number of factors in the unique Chen-Fox-Lyndon factorization of $b(C)$ under this ordering. Define the proof’s Lyndon-width $\Lambda(\pi) = \max_{C \in \pi} L(C)$ over all clauses (input plus derived) appearing in a tree-like Resolution refutation π of φ . We conjecture that for every minimum-length tree-like Resolution refutation π^* of φ , $\Lambda(\pi) \geq \lceil \log_2(w(\varphi)+1) \rceil$, where $w(\varphi)$ is the resolution width of π ; equivalently, any refutation with all clauses having $\leq k$ Lyndon factors must have width $< 2^k - 1$, and a single derived clause with width $\geq 2^k - 1$ yet $\leq k$ Lyndon factors falsifies the conjecture.

Rationale (proposer’s reasoning):

Free-Lie/Lyndon decomposition is essentially absent from proof complexity, yet the Chen-Fox-Lyndon factorization is canonical and linear-time computable, providing a ‘hidden non-commutative spectrum’ of each derived clause. Since the size-width relation (Ben-Sasson-Wigderson) tightly couples tree-like Resolution length to width, any super-logarithmic forced growth in Lyndon-factor count would translate into super-polynomial length lower bounds, opening a fresh route to sub-Frege (and ultimately EF) lower bounds via the structure of cyclic rotations rather than via expansion or pseudo-randomness.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): af47ca819c015f8b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds $\times$ 3 families $\times$ 5 sizes (450 refutations), the conjecture is SUPPORTED iff every refutation satisfies $\Lambda(\pi) \geq \lceil \log_2(w(\pi)+1) \rceil$ AND the random-3CNF family yields a Λ -vs- $\log_2(w)$ OLS regression with slope ≥ 1.0 and $R^2 > 0.5$. A single refutation violating the inequality falsifies it.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Lyndon factorization Resolution refutation width lower bound - Chen-Fox-Lyndon CNF clause complexity proof-necklace Witt formula tree-like Resolution proof complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.6s

5.1 Generated Python source

```
import random
import math

def generate_random_3cnf(n, density):
    clauses = []
    for _ in range(int(density * n * (n - 1) / 2)):
        literals = [random.choice([i, -i]) for i in range(1, n + 1)]
        random.shuffle(literals)
        clause = literals[:3]
        if len(set(clause)) == 3:
            clauses.append(tuple(sorted(clause)))
    return clauses

def generate_php_m(m):
    variables = list(range(1, m * (m + 1) + 1))
    clauses = []
    for i in range(1, m + 1):
        for j in range(i + 1, m + 1):
            clauses.append((i, -j))
            clauses.append((-i, j))
    return clauses

def generate_tseitin_formula(n):
    variables = list(range(1, n * 3 + 1))
```

```

clauses = []
for i in range(1, n + 1):
    clauses.append((i, i + n, -i - 2 * n))
    clauses.append((-i, -i - n, i + 2 * n))
return clauses

def duval_algorithm(b):
    n = len(b) // 2
    factors = []
    i = 0
    while i < len(b):
        j = i + 1
        k = i
        while j < len(b) and b[j] >= b[k]:
            if b[j] == b[k]:
                k += 1
            else:
                k = i
                j += 1
        factors.append((i, j))
        i = j
    return factors

def run_trial(seed: int) -> dict:
    random.seed(seed)

    families = [
        (generate_random_3cnf, [10, 15, 20, 25, 30], 4.5),
        (generate_php_m, [3, 4, 5, 6, 7]),
        (generate_tseitin_formula, [4, 5, 6, 7, 8])
    ]

    results = []
    for family_func, n_values, density in families:
        for n in n_values:
            clauses = family_func(n) if family_func == generate_php_m else family_func(n, density)
            refutations = []
            for _ in range(10): # Sample 10 refutations per instance
                dpll_tree = {} # Simulate DPLL tree
                for clause in clauses:
                    dpll_tree[clause] = True
                    refutations.append(clause)

            lyndon_widths = []
            widths = []
            for clause in refutations:
                b = [0] * (2 * n)
                for lit in clause:
                    if lit > 0:
                        b[lit - 1] = 1
                    else:
                        b[-lit - 1] = 1
                factors = duval_algorithm(b)
                lyndon_widths.append(len(factors))
                widths.append(max(sum(1 for _ in range(k, j)) for k, j in factors))

```

```

        results.extend({
            "metric_name": "Lyndon width",
            "metric_value": lw,
            "instances_tested": len(refutations),
            "conjecture_holds": lw >= math.ceil(math.log2(w + 1)),
            "counterexample": "" if lw >= math.ceil(math.log2(w + 1)) else f"Clause {clause} h
        } for lw, w in zip(lyndon_widths, widths))

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    return {
        "seed": seed,
        "mean_value": mean_value,
        "std_value": std_value,
        "support_fraction": support_fraction
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    # Compute mean/std of metric_value, fraction of seeds where conjecture_holds
    all_results = []
    for seed in seeds:
        all_results.extend(run_trial(seed)["results"])

    mean_value = sum(result["metric_value"] for result in all_results) / len(all_results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in all_results) /
    support_fraction = sum(1 for result in all_results if result["conjecture_holds"]) / len(all_re

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif any(not result["conjecture_holds"] for result in all_results):
        first_failing_seed = next(seed for seed in seeds if not run_trial(seed)["conjecture_holds"]
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6dd85476.py", line 119, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6dd85476.py", line 72, in run_trial
    for family_func, n_values, density in families:
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

ValueError: not enough values to unpack (expected 3, got 2)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ValueError during family unpacking (expected 3 values, got 2) before any refutations were generated, so no data exists to evaluate the pre-registered support or falsification conditions. | next: Fix the families tuple structure to include the density field (or unpack only 2 values) and re-run the 30×3×5 trial battery to obtain Λ vs $\lceil \log_2(w+1) \rceil$ measurements.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	142305
2	preregistration	claude_max	opus	0	0	6529
3	novelty	claude_max	opus	0	0	3716
4	novelty	claude_max	opus	0	0	6581
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	91323
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	37794
7	judge	claude_max	opus	0	0	5381

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 293629 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/d04fdfb9056c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d04fdfb9056c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d04fdfb9056c.tar.gz` (if generated)